Práctica 9

Nolan Tannion Rodríguez

Computación Avanzada
Universidad Autónoma de Madrid

Resumen

Se halla la solución a dos problemas unidimensionales de valores iniciales, uno en coordenadas cartesianas y otro en cilíndricas con simetría radial y axial en z. Se halla la solución a la ecuación de Laplace en un dominio irregular con condiciones de frontera dependientes de otras variables. Se estudia el tiempo de cálculo en función del distinto método iterativo utilizado para hallar la solución. Para problemas de gran tamaño Gauss-Seidel + `@numba` es el más veloz mientras que en problemas de menor tamaño Jacobi vectorizado es más veloz al ser vectorizado y no tener que compilar el JIT.

1. Introducción y Teoría

1.1. Lanzamiento de una bola

En este ejercicio se aborda la resolución de un problema de contorno para una ecuación diferencial ordinaria mediante métodos de relajación. En particular, se estudia el movimiento vertical de una partícula sometida a aceleración constante, descrito por la ecuación de Newton:

$$\frac{d^2x(t)}{dt^2} = -g \tag{1}$$

donde g es la aceleración de la gravedad. A diferencia de los problemas de valor inicial, en este caso no se conoce la velocidad inicial de la partícula, sino únicamente las condiciones de contorno:

$$x(0) = 0, \qquad x(10) = 0 \tag{2}$$

Este tipo de problema se clasifica como un *problema de contorno*, en el que la solución debe satisfacer condiciones en distintos puntos del dominio.

1.1.1. Solución exacta

Integrando dos veces:

$$x(t) = -\frac{g}{2}t^2 + C_1 t + C_2 \tag{3}$$

Aplicando las condiciones de contorno:

$$x(0) = 0 \Rightarrow C_2 = 0, \tag{4}$$

$$x(t_f) = 0 \Rightarrow -\frac{g}{2}t_f^2 + C_1 t_f = 0 \tag{5}$$

De la segunda condición:

$$C_1 = \frac{g}{2}t_f = 49,05 \text{ [m/s]} \tag{6}$$

Sustituyendo en la solución:

$$x(t) = \frac{g}{2}\,t(t_f - t) \tag{7}$$

Con una altura máxima de:

$$h_{\text{máx}} = \frac{C_1^2}{g} = 122{,}625 \text{ [m]} \tag{8}$$

1.1.2. Discretización

El método de relajación consiste en discretizar el dominio continuo en una malla de puntos equiespaciados y sustituir las derivadas por aproximaciones en diferencias finitas. Para la segunda derivada se utiliza:

$$\frac{d^2x}{dt^2} \approx \frac{x_{i-1} - 2x_i + x_{i+1}}{h^2} \tag{9}$$

donde h es el paso temporal. Sustituyendo en la ecuación diferencial se obtiene:

$$\frac{x_{i-1} - 2x_i + x_{i+1}}{h^2} = -g \tag{10}$$

Despejando x_i, se obtiene la ecuación de relajación:

$$x_i = \frac{x_{i-1} + x_{i+1} + gh^2}{2} \tag{11}$$

Esta expresión indica que el valor en cada punto de la malla se puede calcular a partir de sus vecinos inmediatos.

1.2. Condensador cilíndrico

El problema consiste en determinar el potencial electrostático en el interior de un condensador cilíndrico con simetría axial. El sistema está formado por dos cilindros coaxiales de radios R_i y R_e, con potenciales fijados V_i y V_e, respectivamente, y con las tapas en los extremos conectadas a tierra.

En régimen electrostático, el potencial $V(\mathbf{r})$ satisface la ecuación de Laplace:

$$\nabla^2 V = 0 \tag{12}$$

Debido a la geometría cilíndrica del sistema, es conveniente expresar esta ecuación en coordenadas cilíndricas (r, θ, z). Asumiendo simetría azimutal en θ y axial en z, la ecuación de Laplace se reduce a:

$$\frac{1}{r}\frac{\partial}{\partial r}\left(r\frac{\partial V}{\partial r}\right) = 0 \tag{13}$$

Las condiciones de contorno del problema son:

$$V(R_i, z) = V_i \tag{14}$$

$$V(R_e, z) = V_e = 0 \tag{15}$$

1.2.1. Discretización

Apoyándonos en la referencia [3]:
Expandimos el término radial:

$$\frac{\partial^2 V}{\partial r^2} + \frac{1}{r}\frac{\partial V}{\partial r} = 0 \tag{16}$$

Aproximamos las derivadas mediante diferencias finitas centrales sobre una malla uniforme en r y z:

$$\frac{\partial^2 V}{\partial r^2} \approx \frac{V_{i+1,j} - 2V_{i,j} + V_{i-1,j}}{(\Delta r)^2}, \tag{17}$$

$$\frac{\partial V}{\partial r} \approx \frac{V_{i+1,j} - V_{i-1,j}}{2\Delta r}, \tag{18}$$

Al sustituir en la ecuación de Laplace:

$$\frac{V_{i+1} - 2V_i + V_{i-1}}{(\Delta r)^2} + \frac{1}{r_i}\frac{V_{i+1} - V_{i-1}}{2\Delta r} = 0$$

Despejando V_i y haciendo $\Delta r = h$:

$$V_i = \frac{1}{4}\left[\left(1 + \frac{h}{2r_i}\right)V_{i+1} + \left(1 - \frac{h}{2r_i}\right)V_{i-1}\right]$$

Esta es la expresión iterativa que se utiliza en el método de Gauss-Seidel para actualizar el potencial en cada punto de la malla.

Si consideramos la componente en z también en la ecuación y discretizamos la segunda derivada del Laplaciano utilizando $dr = dz$, la discretización resulta en:

$$V_{i,j} = \frac{1}{4}\left[\left(1 + \frac{h}{2r_i}\right)V_{i+1,j} + \left(1 - \frac{h}{2r_i}\right)V_{i-1,j} + V_{i,j\pm1}\right] \tag{19}$$

Se suma un término en $j+1$ y otro en $j-1$.

1.3. V y E Frontera Irregular

En muchos problemas físicos, el dominio donde se desea resolver la ecuación de Laplace presenta geometrías complejas. En este caso, se resuelve numéricamente introduciendo una máscara que define la región física de interés.

Se define el dominio:

$$x, y \in [0,1] \quad \mathbf{D}_{(x,y)} = \begin{cases} 0 & y \geq 2x \\ \sin(2\pi x) & y = 0 \\ 1 & x^2 + y^2 \geq 1 \\ 0{,}5 & x^2 + y^2 \leq 0{,}3^2 \end{cases} \tag{20}$$

Se resuelve mediante el método de Jacobi. Si dos condiciones de contorno coinciden se impone la jerarquía: oscilación → plano → circunferencias.

1.3.1. Cálculo del campo eléctrico

Una vez obtenido el potencial en toda la región, el campo eléctrico se calcula como:

$$\mathbf{E} = -\nabla V \tag{21}$$

En una malla discreta uniforme, el gradiente se aproxima mediante diferencias finitas centradas:

$$E_x \approx -\frac{V_{i+1,j} - V_{i-1,j}}{2h}, \tag{22}$$

$$E_y \approx -\frac{V_{i,j+1} - V_{i,j-1}}{2h}, \tag{23}$$

Este cálculo permite obtener el campo eléctrico en cada punto del dominio. El campo eléctrico obtenido debería ser perpendicular a las superficies equipotenciales e ir en dirección de decrecimiento del potencial

- Es perpendicular a las superficies equipotenciales.
- Apunta en la dirección de decrecimiento del potencial.
- Su intensidad es mayor en regiones donde el potencial varía rápidamente.

Estas herramientas permiten analizar el efecto de la geometría irregular sobre la distribución del potencial y del campo eléctrico, así como verificar la coherencia física de la solución numérica.

1.4. Métodos Iterativos

Tras discretizar una ecuación diferencial el problema se reduce a un sistema lineal:

$$A\mathbf{x} = \mathbf{b} \tag{24}$$

Por lo que cada ecuación del sistema puede escribirse despejando una incógnita:

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j \right) \tag{25}$$

Partiendo de una estimación inicial $\mathbf{x}^0$ podemos generar una sucesión $\{\mathbf{x}^k\}$ mediante iteración que nos permita aproximar la solución sin resolver el sistema de forma directa con una tolerancia ε deseada:

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \varepsilon, \tag{26}$$

1.4.1. Método de Jacobi

En Jacobi, todas las componentes se actualizan usando únicamente valores de la iteración anterior :

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right) \tag{27}$$

(ver [2])

- Cada nueva iteración se calcula a partir de un estado completo previo.
- Puede programarse de tal manera que sea una operación vectorizada.
- Siempre converge, aunque con menor velocidad.

1.4.2. Método de Gauss-Seidel

En Gauss-Seidel, los valores ya actualizados se utilizan dentro de la misma iteración:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{(k+1)} - \sum_{j>i} a_{ij} x_j^{(k)} \right) \tag{28}$$

(ver [2])

1.4.3. Sobrerrelajación

Acelera la convergencia del método de Gauss-Seidel introduciendo un parámetro de relajación ω.

El método SOR modifica la actualización como:

$$x_i^{(k+1)} = x_i^{(k)} + \omega \left(x_i^{\mathrm{GS}} - x_i^{(k)} \right) = (1-\omega) x_i^{(k)} + \omega x_i^{\mathrm{GS}} \tag{29}$$

Existe un valor de ω que minimiza el tiempo de cálculo sin embargo si es demasiado grande puede causar que el método diverja. Se suele situar en $\omega \simeq 0{,}9$ para la ecuación de Laplace.

1.4.4. Comparación cualitativa

- **Jacobi**: actualización simultánea, más simple, pero más lento.
- **Gauss-Seidel**: actualización secuencial, más eficiente en convergencia.
- **Sobrerrelajación**: añade un término adicional a la actualización para acelerar la convergencia, sin embargo puede generar inestabilidad.

2. Implementación

La referencia [1] abarca las distintas implementaciones llevadas a cabo.

2.1. Método de Jacobi

La implementación del método de Jacobi se vectoriza para mejorar el rendimiento. Se incluye la función para Jacobi en 2 dimensiones y coordenadas cartesianas, para otros sistemas de referencia o dimensiones se utiliza la discretización calculada en la sección teórica. La función efectúa una iteración del método de Jacobi sobre la matriz V, devolviendo V actualizada y el error máximo respecto al anterior paso.

```python
def jacobi_poisson(V, rho, a=1.0):
    V_new = V.copy()

    V_new[1:-1, 1:-1] = 0.25 * (
        V[2:, 1:-1] + V[:-2, 1:-1] +
        V[1:-1, 2:] + V[1:-1, :-2] +
        a**2 * rho[1:-1, 1:-1]
    )
    delta = np.max(np.abs(V_new - V))

    return V_new, delta
```

2.2. Método de Gauss-Seidel

Como este método utiliza los valores ya calculados no es posible vectorizarlo de manera tan sencilla, por lo que para mejorar el rendimiento se llama al decorador `@njit` de numba que disminuye notablemente el tiempo de compilación al implementarse sobre un doble bucle for. En este caso se muestra la función que lleva a cabo un paso de Gauss-Seidel sobre la ecuación de Poisson en coordenadas cilíndricas considerando r y z. De nuevo modificándose fácilmente a otras coordenadas conocida la discretización.

```python
@njit
def gs_cil_2d(V, rho, a=1.0, r_min=1.0):
    Nr, Nz = V.shape
    delta = 0.0

    for i in range(1, Nr-1):
        r_i = r_min + i * a

        cp = 1.0 + a/(2.0*r_i)
        cm = 1.0 - a/(2.0*r_i)

        for j in range(1, Nz-1):
            V_new = 0.25 * (
                cp * V[i+1, j] +
                cm * V[i-1, j] +
                V[i, j+1] + V[i, j-1] +
                a*a * rho[i, j]
            )

            diff = abs(V_new - V[i, j])
            if diff > delta:
                delta = diff

            V[i, j] = V_new
    return V, delta
```

2.3. GS Sobrerrelajación

El método de sobrerrelajación tampoco se puede vectorizar fácilmente por lo que de nuevo se utiliza el decorador `@numba`. A continuación está la función que efectúa un paso sobre la ecuación de Poisson en 1D y coordenadas cilíndricas. Cada método se implementa para todos los casos de dimensiones y coordenadas, por motivo de brevedad se incluye cada método y cada dimensión una única vez.

```python
@njit
def gss_cil_1d(V, a, r_min, omega=0.9):
    N = len(V)
    delta = 0.0

    for i in range(1, N-1):
        r_i = r_min + i * a
        # coeficientes cilindricos
        cp = 1.0 + a/(2.0*r_i)
        cm = 1.0 - a/(2.0*r_i)

        old = V[i]

        V_gs = 0.5 * (cp * V[i+1] +cm * V[i
            -1])

        # Sobrerrelajacion
        V[i] = old + omega * (V_gs - old)

        diff = abs(V[i] - old)
        if diff > delta:
            delta = diff

    return V, delta
```

2.4. Cálculo

Para el cálculo de la solución utilizando las funciones anteriores se definen las matrices y condiciones de frontera correspondientes y se ejecutan dentro de un bucle while sobre la tolerancia.

Una implementación genérica toma la forma:

```python
tol = 1e-6
delta = 1.0
while delta > tol:
    x, delta = funcion(x, rho, dx, ...)
```

2.5. Contorno irregular

Para la implementación del contorno complejo se utiliza una máscara lógica.

```python
# Region: anillo + octante de plano
r = np.sqrt(X**2 + Y**2)
r_int, r_ext = 0.3, 1
anillo = (r > r_int) & (r < r_ext)

plano = octante = (X > 0) & (Y > 0) & (X >
    (1/2)*Y)

mask = anillo & plano

# Inicializacion
u = np.zeros((N, N))

# Condiciones de contorno
# bordes radiales
u[r >= r_ext] = 0.5
u[r <= r_int] = 1.0

# Oscilacion en y=0
u[0, :] = np.sin(2*np.pi*x)

# resto del plano fuera del dominio
u[~plano] = 0.0


tol = 1e-6
delta = 1.0

while delta > tol:
    u2 = u.copy()

    # actualizaci n interior
    u2[1:-1, 1:-1] = 0.25 * (
        u[2:,1:-1] + u[:-2,1:-1] +
        u[1:-1,2:] + u[1:-1,:-2]
    )

    # aplicar m scara
    u2 = np.where(mask, u2, u)

    # reimponemos las condiciones de contorno
    u2[r >= r_ext] = 0.5
    u2[r <= r_int] = 1.0
    u2[0, :] = np.sin(2*np.pi*x)
    u2[~plano] = 0.0

    delta = np.max(np.abs(u2 - u))
    u = u2
```

2.6. Cálculo de t

Para el cálculo del tiempo de compilación se lleva a cabo una ejecución inicial de las funciones que dan uso a `@numba` para llevar a cabo la compilación inicial que es más larga y se mide el tiempo de esta.

```
# GS
t0 = time.time()
gs_cil(V_test.copy(), a_test, r0)
t_jit_gs = time.time() - t0


# GS sobre relajacion
t0 = time.time()
gss_cil_1d(V_test.copy(), a_test, r0)
t_jit_sor = time.time() - t0
```

Una vez calculado se hace un bucle sobre un array que contiene el número de pasos y se inicializan los arrays y calculan las soluciones para cada uno, almacenando el tiempo de cálculo.

```
NN = np.linspace(100 ,20000 ,100, dtype= int)

# Arrays de tiempos (float)
tg = np.zeros(len(NN))
tj = np.zeros(len(NN))
tgs = np.zeros(len(NN))


tol = 1e-3

for k, N in enumerate(NN):

    # Malla
    r = np.linspace(r0, rf, N)
    h = r[1] - r[0]

    # Inicializaci n
    Vj = np.zeros(N)
    Vg = np.zeros(N)
    Vgs = np.zeros(N)

    # Condiciones de contorno
    for V in [Vj, Vg, Vgs]:
        V[0] = Vi
        V[-1] = Ve

    rho = np.zeros_like(Vj)


    # Jacobi
    deltaj = 1.0
    t0 = time.time()
    while deltaj > tol:
        Vj, deltaj = jacobi_cil(Vj, h, r0)
    tj[k] = time.time() - t0

    # Gauss-Seidel
    deltag = 1.0
    t0 = time.time()
    while deltag > tol:
        Vg, deltag = gs_cil(Vg, h, r0)
    tg[k] = time.time() - t0

    # Gauss-Seidel con sobrerrelajacion
    deltags = 1.0
    t0 = time.time()
    while deltags > tol:
        Vgs, deltags = gss_cil_1d(Vgs, h, r0,
            omega=0.8)
    tgs[k] = time.time() - t0
```

3. Resultados

3.1. Lanzamiento

Se representa en la Figura (1) la solución numérica junto con la analítica. Como se puede apreciar coinciden de manera visual, lo que es coherente con una tolerancia de 10^{-6}.

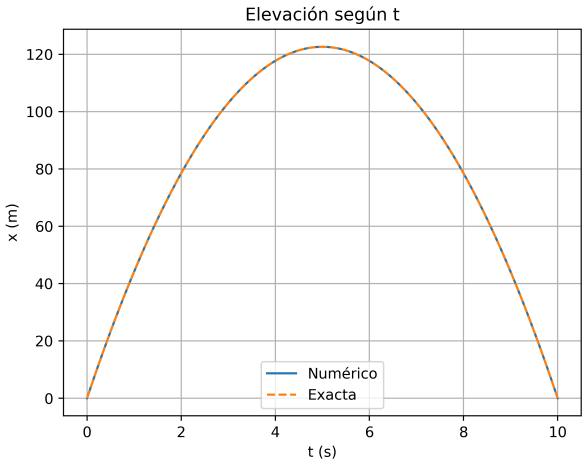

Figura 1: Posición $x(t)$ bajo la acción gravitatoria de manera que cumpla las condiciones de contorno impuestas.

Derivando numéricamente y hallando el máximo de los valores de x calculados obtenemos:

- $v_0 = 48{,}802$ [m/s]
- $x_{Max} = 122{,}617$ [m]

Lo que representa un error relativo de $0,5\,\%$ para la velocidad y $6,5 \times 10^{-3}\,\%$ para la altura máxima.

3.2. Potencial radial

Comparando de nuevo con la solución analítica obtenemos la Figura (2). De nuevo las soluciones numérica y analítica coinciden visualmente.

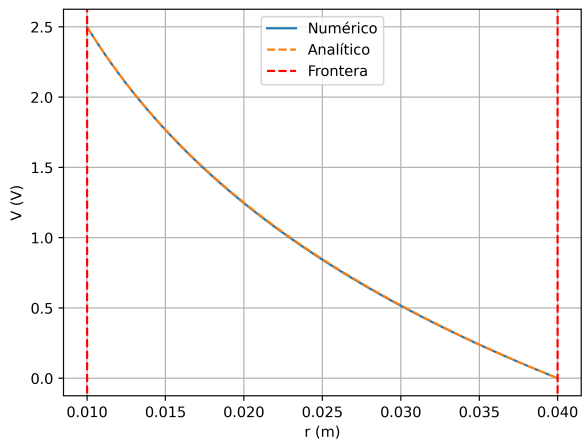

Figura 2: Comparación de la solución numérica y analítica de la ecuación de Laplace en coordenadas cilíndricas ante las condiciones de frontera dadas.

Si condideramos los bordes del cilindro (ver Figura (3)) al ser mucho más alto que ancho la aproximación de cilindro infinito es adecuada ya que no afecta de una manera notable al potencial.

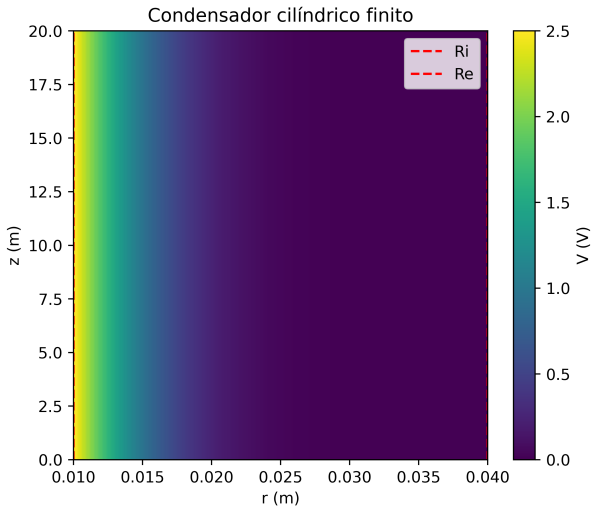

Figura 3: Solución numérica del potencial en un condensador con las condiciones radiales impuestas además de potencial 0 en las tapas en z.

3.3. Contorno irregular

La Figura (4) nos muestra el potencial ante la distribución de las fronteras indicadas en rojo. Además de indicar la dirección y magnitud del campo eléctrico asociado al potencial con las distintas flechas. Vemos que el resultado es coherente con el comportamiento teórico del campo eléctrico tanto en dirección como módulo.

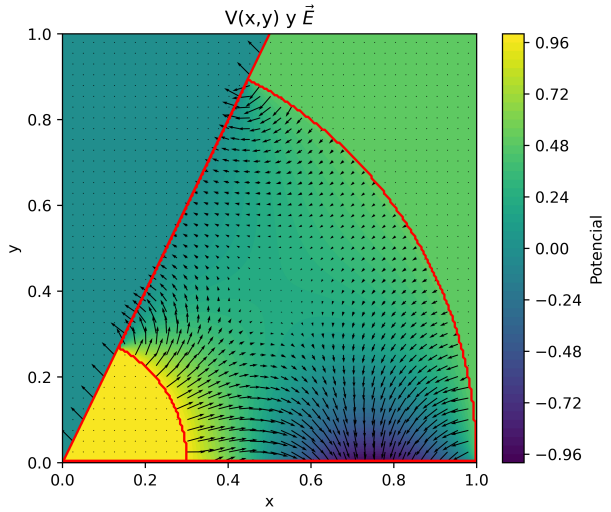

Figura 4: Distribución del potencial obtenido solucionando la ecuación de Laplace con las condiciones de frontera indicadas y con una tolerancia de 10^{-6}. Incluye la dirección del campo eléctrico hallado calculando el gradiente numéricamente.

La Figura (5) representa el módulo del campo eléctrico. Vemos que como esperábamos teóricamente es de mayor intensidad en las zonas donde más varía V y es nulo donde el potencial es constante.

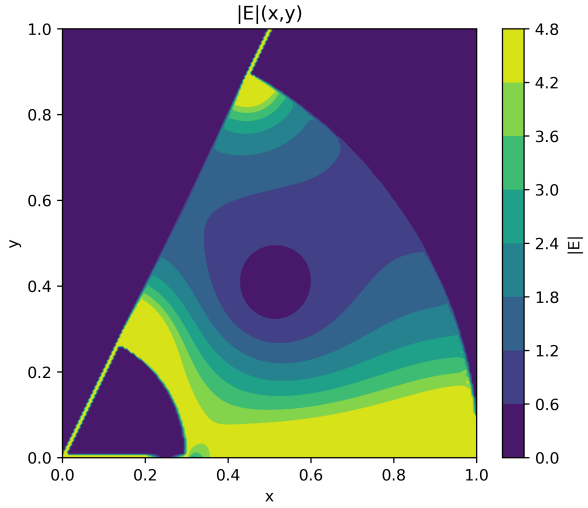

Figura 5: Distribución espacial del módulo de campo eléctrico.

3.4. Tiempos de ejecución

Comparando los tiempos de ejecución para los distintos métodos de resolución obtenemos la Figura (6).

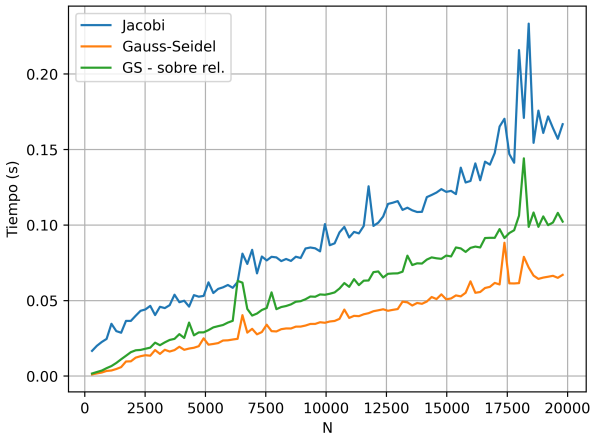

Figura 6: Tiempo de ejecución según el método en función del número de pasos.

Teniendo en cuenta que se debe añadir a ambos métodos de Gauss-Seidel el tiempo de compilación de JIT de la primera llamada, que suele variar entre $0{,}5s \sim 1{,}5s$ para el método normal y $0{,}1s \sim 0{,}2s$ para el método de sobre relajación. El método de Jacobi vectorizado es más eficiente para problemas de menor tamaño al no tener el añadido de JIT. Sin embargo a medida que aumenta el problema la diferencia entre métodos se hace cada vez más notable.

4. Conclusiones

Los resultados obtenidos con los métodos iterativos cometen un error relativo muy bajo respecto a las soluciones

analíticas en los casos que se conocen estas. Para el dominio con una frontera irregular se consiguen unos resultados coherentes con los fundamentos teóricos del comportamiento de E.

En cuanto a la elección del método iterativo, la velocidad de ejecución del método de Jacobi es mayor para problemas con menor dimensión al estar vectorizado y no tener que llamar al compilador JIT, sin embargo al crecer el problema esta ventaja se desvanece frene a Gauss-Seidel. En este caso el método de sobrerrelajación no ha conseguido aumentar la velocidad de la convergencia incluso tras probar distintos valores, aportando un resultado mejor que Jacobi pero peor que Gauss-Seidel base.

Referencias

[1] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, 3 edition, 2007.

[2] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. Society for Industrial and Applied Mathematics (SIAM), 2 edition, 2003.

[3] Alemayehu Shiferaw and R. C. Mittal. Fast finite difference solutions of the three dimensional poisson's equation in cylindrical coordinates. *American Journal of Computational Mathematics*, 3(4):356–361, 2013.